ARCH-1R/T Operational Reachability Framework

Expanded review manuscript with formal math first, plain-language analysis, charts, and worked examples

Prepared for John Haines / Chaos Rider
Draft Review Version 2.0 - June 2026

Purpose of this version

This version replaces the earlier bullet scaffold with a real review document. The first half presents the mathematical specification up front. The second half translates the framework into plain language, examples, engineering implications, and review questions.

Abstract

ARCH-1R/T is a layered reachability and coherence framework built on the original ARCH-1 recursive kernel. The original kernel, Sigma := mu S.(beta OR Omega(S,S)), generates a structured possibility space from an anchor beta and binary constructor Omega. ARCH-1R/T does not replace this kernel. It adds operational layers that answer what happens after possibility is generated: what is traversed, what is obstructed, what remains realizable, what is latent, what is observed, and how prediction diverges from reality.

The framework introduces a ternary reachability classifier tau:T->{+,0,-} that distinguishes reachable, latent, and obstructed states. This distinction is the central practical advance: many systems incorrectly treat waiting, pending, unresolved, or dependency-bound states as failures. ARCH-1T represents these as latent states rather than obstructed states. The result is a general model for workflow governance, UOR-relational systems, runtime orchestration, planning, dependency cascades, and validation through coherence error Delta.

Reader Guide

Section	Audience	Purpose
Part I - Formal Specification	Mathematical and technical reviewers	Defines the exact objects, equations, transition rules, and validation layer.
Part II - Worked Examples	Systems engineers and UOR reviewers	Demonstrates the framework on Boolean logic, governance, dependencies, relations, and planning.
Part III - Analysis	Reviewers and decision makers	Explains first-, second-, and third-order effects and external alignment.
Part IV - Implementation and Review	Builders and evaluators	Provides pseudocode, falsification criteria, simulation design, and questions for critique.

Part I - Formal Specification

1. Original Kernel Preserved

The starting point is the original ARCH-1 recursive construction. This remains the foundation and is not modified by the later reachability extensions.

```
Sigma := mu S.(beta OR Omega(S,S))

G := Closure_Omega(beta)
```

Interpretation: beta is the terminal anchor or seed; Omega is the binary recursive constructor; mu denotes least fixed-point closure; G is the generated possibility space. The kernel answers the question: what can be structurally generated?

Core alignment rule

All later operators are downstream of G. They do not redefine beta, Omega, or mu. This preserves the original ARCH-1 lineage.

2. Domain Instantiations of the Kernel

Domain	Anchor beta	Constructor Omega	Generated space	Status
Boolean	x	NAND	Boolean expressions built from NAND	Strongest formal case. NAND functional completeness is established.
Continuous / EML	1	eml	Constructive continuous expression space	Research instantiation. Useful, but completeness requires proof.
Sovereignty / Blade	null	blade	Action or agency path expressions	Symbolic and operational hypothesis. Not yet a theorem.
Runtime systems	initial state	action constructor	Reachable runtime state space	Engineering instantiation. Testable by simulation and replay.
UOR-relational	entity/relation seed	relation constructor	Generated relational structure	Philosophically aligned and operationally modelable.

The careful claim is not that these domains are identical. The claim is that they can be described through a shared architecture of anchor, binary constructor, and recursive closure.

3. Traversal Layer

Generation alone is not execution. A generated possibility may exist structurally without ever being selected, attempted, scheduled, or explored. The traversal layer separates generated possibility from attempted or activated possibility.

```
rho : G -> T
T := rho(G)
```

rho may represent a scheduler, planner, execution policy, proof search procedure, governance routing process, agent policy, or relational traversal rule. This layer answers the question: what part of generated possibility is actually explored?

4. Obstruction Layer

Obstruction is any condition that prevents a traversed state or relation from becoming realizable. Obstruction may be a constraint, dependency failure, missing permission, contradiction, unavailable resource, policy boundary, or invalid transition.

```
O subseteq T
R := T \ O
```

R is the realizable space after direct obstruction. This equation is intentionally simple: generated and traversed possibility are not the same as realized possibility.

5. Dependency Closure and Propagated Obstruction

Real systems rarely fail locally. Obstruction often propagates through dependencies. Let D be a dependency graph whose edges represent prerequisite, authority, resource, or causal dependence.

```
D = (V,E)
O* := closure_D(O)
R := T \ O*
```

O* is the propagated obstruction set. This is the basis for cascade analysis. If x is obstructed and y depends on x, then y may become latent or obstructed depending on the dependency rule. A hard dependency propagates obstruction. A soft or redundant dependency may only create latency.

Dependency type	Propagation effect	Example
Hard dependency	Obstruction propagates directly.	A payment cannot execute without required authorization.
Soft dependency	Downstream state becomes latent.	A review can continue but final approval waits.
Redundant dependency	Obstruction propagates only if all paths fail.	A service can use backup provider B if provider A fails.
Optional dependency	No obstruction propagates unless required by policy.	A report can be published without optional appendix.

6. Ternary Reachability

Binary reachability is too coarse for operational systems. ARCH-1T adds a ternary classifier over the traversed space.

```
tau : T -> {+,0,-}
```

Class	Name	Definition	Diagnostic meaning
+	Reachable	Realizable under current conditions.	The state can execute, the relation is active, or the path is open.
0	Latent	Not currently realized, but not impossible.	The state is waiting, pending, unresolved, or dependency-bound.
-	Obstructed	Blocked by active constraint.	The state is denied, contradicted, broken, forbidden, or invalid.

```
T = T_plus union T_zero union T_minus
T_plus intersection T_zero = empty
T_plus intersection T_minus = empty
T_zero intersection T_minus = empty
```

Fundamental ternary law

0 != -. A latent state is not an obstructed state. Not yet is not the same as impossible.

7. Transition Algebra

Once the ternary classes exist, the framework becomes dynamic. Classification can change as information, dependencies, resources, decisions, or constraints change.

Transition	Name	Formal meaning	Practical example
0 -> +	Resolution	Latent becomes reachable.	A pending approval is granted.
+ -> 0	Uncertainty	Reachable becomes unresolved.	A requirement changes and the path needs review.
0 -> -	Obstruction	Latent becomes blocked.	A review returns denial or contradiction.
- -> 0	Relief	Obstructed becomes latent.	A blocking policy is amended, but final approval still waits.
+ -> -	Hard block	Reachable becomes blocked.	Permission is revoked.
- -> +	Bypass or direct remediation	Obstructed becomes reachable through alternate path.	A backup route fully replaces the failed dependency.

This transition algebra is intentionally minimal. It is enough to model delay, relief, denial, approval, runtime waiting, dependency recovery, and governance state changes without requiring probability yet.

8. Coherence and Validation

A model becomes useful only when it can be compared to observation. Let Q be the observed outcome set and d be a domain-specific distance metric.

```
Q := observed outcomes
d : R x Q -> Real_nonnegative
Delta := d(R,Q)
```

Delta measures mismatch between predicted realizability and observed reality. Persistent high Delta is a failure signal. It may indicate hidden obstruction, wrong dependency structure, bad traversal, incomplete generation, telemetry error, or a poor metric.

9. Unified Equation

```
G := Closure_Omega(beta)
T := rho(G)
O* := closure_D(O)
R := T \ O*
tau : T -> {+,0,-}
Q := observed outcomes
Delta := d(R,Q)

Compact form:
Delta := d(rho(Closure_Omega(beta)) \ closure_D(O), Q)
```

10. Proposed Quantification and Cost Layers

The current ternary system is qualitative. A natural next step is to add a reachability score and cost function while preserving the ternary categories.

```
sigma : T -> [0,1]

+ if sigma(x) > theta_plus
0 if theta_minus <= sigma(x) <= theta_plus
- if sigma(x) < theta_minus

C : T -> Real_nonnegative
P_lambda := { x in R | C(x) <= lambda }
```

sigma supports graded reachability. C supports practical reachability: a state may be reachable but too expensive, too slow, too risky, or too resource-intensive.

Part II - Worked Examples

11. Example A - Boolean NAND

The Boolean instantiation is the cleanest formal example. Let beta=x and Omega=NAND. The recursive grammar is:

```
S -> x | NAND(S,S)
```

This generates expressions using NAND alone. Because NAND is functionally complete, Boolean negation, conjunction, disjunction, and all Boolean functions can be reconstructed using only NAND.

Construction	Boolean result	Explanation
NAND(x,x)	NOT x	Self-application yields inversion.
NAND(NAND(x,y), NAND(x,y))	x AND y	NAND result is inverted by self-application.
NAND(NAND(x,x), NAND(y,y))	x OR y	Negate each input and combine through NAND.

ARCH-1T does not claim that NAND expressions are inherently positive, neutral, or negative. Ternary classification is applied later by a proof process, evaluator, satisfiability procedure, or runtime context. This preserves the original kernel.

12. Example B - Governance Approval

A governance process often has the exact ternary structure ARCH-1T was designed to preserve: approved, pending, and denied. Consider a proposed action requiring legal, budget, and executive approval.

```
request -> legal_review -> budget_review -> executive_approval -> execution
```

Step	State	ARCH-1T class	Meaning
request submitted	exists	0	The request is alive but not actionable.
legal review pending	waiting	0	Not failure. The process is latent.
legal approval granted	resolved	+ for legal gate	The next dependency can activate.
budget denied	blocked	- for execution path	Downstream execution becomes obstructed.
alternate funding found	relief	- -> 0	The block is relieved but still requires final approval.
executive approval granted	resolved	0 -> +	The action becomes executable.

This example shows why 0 != -. A pending legal review is not denial. Treating it as denial produces false failure reports, premature escalation, and bad operational decisions.

13. Example C - Runtime Dependency Cascade

Suppose a runtime has five services. D and E both depend on B. B depends on A. C is independent.

```
A -> B -> D
A -> B -> E
C
```

If A is obstructed, the direct obstruction is O={A}. Under hard dependency closure, O*={A,B,D,E}. C remains reachable. Under soft dependency rules, B, D, and E may enter latent state rather than hard obstruction if recovery is expected.

Node	Direct status	Hard dependency result	Soft recovery result
A	-	-	-
B	depends on A	-	0
D	depends on B	-	0
E	depends on B	-	0
C	independent	+	+

The framework exposes the difference between actual failure and waiting for recovery. This matters in runtime dashboards, incident management, and automated remediation.

14. Example D - UOR-Relational Structure

UOR-relational interpretation treats relations as primary. Instead of classifying only entities, ARCH-1R/T classifies relations.

```
rel(a,b) in T
tau(rel(a,b)) in {+,0,-}
```

Relation	Class	Interpretation
rel(person, authority)	+	Authority relation is active and operational.
rel(person, pending_authority)	0	Authority relation is requested but unresolved.
rel(person, forbidden_action)	-	The relation is blocked by policy or contradiction.
rel(system, backup_path)	0	Backup exists as potential but is not active.
rel(system, backup_path)	+	Backup has been activated and is now realizable.

This relational view is stronger than object-only modeling because it lets obstruction attach to the relation itself. The person may exist and the action may exist, but the relation authorizing the person to perform the action may be latent or obstructed.

15. Example E - Planning as Latency Conversion

In ARCH-1T, planning can be stated as a process for converting required latent states into reachable states while avoiding obstruction.

```
Goal g in T_zero
Find prerequisites pre(g)
Resolve pre(g): 0 -> +
Then resolve g: 0 -> +
```

Planning stage	ARCH-1T expression	Plain meaning
Goal identified	g in T_zero	The goal is possible but not yet actionable.
Prerequisites found	pre(g) subset T_zero	The required dependencies are pending.
Action selected	rho chooses transition	Traversal picks a path through prerequisites.
Prerequisite resolved	pre(g): 0 -> +	The dependency becomes available.
Goal executed	g: 0 -> +	The goal becomes reachable and is realized.

This reframes planning as latency management. The central question is not only can we reach the goal, but what must move from 0 to + for the goal to become reachable?

16. Example F - Falsification Case

Assume the model predicts that a workflow action is reachable because all required dependencies are marked +. Observation Q shows the action did not execute. Delta increases.

```
Predicted: action in R
Observed: action not in Q
Delta > epsilon
```

The model has failed in a useful way. The failure must be investigated as one of several causes: hidden obstruction, wrong dependency graph, wrong traversal assumption, bad observation, or an incorrect distance metric. This gives reviewers a concrete way to test and improve the framework.

Part III - Analysis and Results

17. First-Order Effects

First-order effects are immediate changes in what the framework can represent.

Effect	Before ARCH-1T	After ARCH-1T
Delay representation	Often treated as failure or absence.	Represented as latent state 0.
Obstruction	Implicit error or missing path.	Explicit object O with propagated O*.
Validation	Often narrative or qualitative.	Measured by Delta=d(R,Q).
Reachability	Binary reachable/unreachable.	Reachable, latent, obstructed.

18. Second-Order Effects

Second-order effects appear when the framework is applied to systems.

System type	ARCH-1R/T improvement
Governance	Separates approved, pending, and denied without collapsing pending into denial.
Runtime orchestration	Distinguishes runnable, waiting, and blocked states.
Dependency management	Computes cascades and identifies single points of propagated obstruction.
Auditing and replay	Explains why an expected path did not execute.
UOR modeling	Classifies relations as active, latent, or obstructed.

19. Third-Order Effects

Third-order effects are conceptual consequences that emerge once the ternary state and obstruction layer are both present.

Effect	Explanation	Status
Potential becomes formal	The neutral state 0 gives a formal home to possible-but-not-yet-realized states.	Strong conceptual result.
Planning emerges	Planning becomes the search for transitions that move required states from 0 to +.	Operationally useful.
Resilience becomes measurable	A system is resilient when required states remain + or recover from 0 despite obstruction.	Needs metrics.
Agency becomes investigable	Agents can be studied as systems that transform 0 into + under constraints.	Hypothesis, not theorem.

20. Alignment With the Original ARCH-1

The current framework remains aligned with the original because classification, obstruction, traversal, and validation are downstream layers. They do not alter the recursive generator.

Original component	Status in ARCH-1R/T	Comment
beta	Preserved	Still the anchor or seed.
Omega	Preserved	Still the binary constructor.
mu	Preserved	Still recursive closure.
G=Closure_Omega(beta)	Preserved	Still the generated possibility space.
rho	Clarified	Interpreted as traversal or activation over G.
O	Added	Constrains T after traversal.
tau	Added	Classifies traversed states.
Delta	Added	Validates prediction against observation.

21. What Is Actually New

The components of ARCH-1R/T are not individually unprecedented. Recursion, state transitions, dependencies, constraints, and validation metrics all exist in related fields. The distinctive contribution is their synthesis into a single lineage that begins with recursive generation and proceeds through traversal, obstruction, ternary classification, and coherence validation.

One-sentence result

ARCH-1R/T is a reachability and coherence calculus built on recursive generation, where the most important operational distinction is that latent is not obstructed.

22. Weaknesses and Current Limits

Limit	Why it matters	Next fix
No built-in cost model	Reachable and expensive are treated the same.	Add C:T->Real and practical reachability P_lambda.
No built-in probability	Rare and likely paths are treated the same.	Add probabilistic traversal or confidence scoring.
Single rho is too simple	Distributed systems may use many traversal engines.	Use rho_i and composition rules.
Obstruction transformations are early	Relief, bypass, displacement, and inversion need formal laws.	Develop obstruction algebra v2.
Distance metric is unspecified	Delta requires domain-specific definition.	Define metric families by domain.

23. Review Assessment

Area	Score	Reason
Internal consistency	Strong	The layers are ordered and preserve the original kernel.
Engineering utility	Very strong	Maps naturally to workflows, runtime systems, dependencies, and audits.
UOR-relational fit	Strong	Relations can be classified directly as active, latent, or obstructed.
Mathematical novelty	Moderate	The novelty is synthesis and application, not each component alone.
Publication readiness	Concept paper ready	Needs related work and simulations for manuscript-level review.

Part IV - Implementation, Tests, and Review Path

24. Minimal Simulation Design

A minimal simulation can test whether ARCH-1R/T provides more diagnostic value than binary reachability.

```
Input:
  Directed graph D
  Initial classification tau(x) for each node
  Obstruction set O
  Dependency policy: hard, soft, redundant, optional
  Observed outcome Q

Procedure:
  1. Compute O* = closure_D(O)
  2. Compute R = T \ O*
  3. Update tau over T
  4. Compare R against Q
  5. Report Delta and misclassification cases

Expected result:
  The ternary model reduces false failure classification by separating waiting states from
blocked states.
```

25. Pseudocode Reference

```
def classify_state(x, context):
    if hard_obstruction_exists(x, context):
        return '-'
    if prerequisites_pending(x, context):
        return '0'
    if executable_now(x, context):
        return '+'
    return '0'

def dependency_closure(obstructions, graph, policy):
    closed = set(obstructions)
    frontier = list(obstructions)
    while frontier:
        node = frontier.pop()
        for child in graph.downstream(node):
            if policy.propagates(node, child):
                if child not in closed:
                    closed.add(child)
                    frontier.append(child)
    return closed

def arch1rt_step(beta, omega, rho, graph, O, Q, metric):
    G = closure(beta, omega)
    T = rho(G)
    O_star = dependency_closure(O, graph, policy='domain-specific')
    R = T - O_star
    labels = {x: classify_state(x, context=(T, O_star, graph)) for x in T}
    Delta = metric(R, Q)
    return G, T, O_star, R, labels, Delta
```

26. Falsification Criteria

Failure case	Formal signal	Likely cause
Predicted reachable, observed absent	x in R but x not in Q	Hidden obstruction, bad traversal, bad observation.
Observed state outside generated space	x in Q but x not in G	Incomplete beta/Omega model.
Latent misclassified as blocked	x marked - but resolves without structural change	Need better ternary classification.
Cascade missed	Downstream failure not in O*	Dependency graph incomplete.
Persistent high Delta	Delta > epsilon over repeated trials	Framework model not adequate for the domain.

27. Proposed Roadmap

1. Lock the notation hierarchy: G, T, O*, R, tau, Q, Delta.
2. Define obstruction transformation laws: relief, bypass, displacement, inversion, and cascade damping.
3. Implement a toy graph simulator with hard, soft, redundant, and optional dependencies.
4. Compare binary reachability against ternary reachability on false failure classification.
5. Add sigma scoring and cost C only after the ternary model is stable.
6. Prepare a related-work review against state machines, Petri nets, planning systems, dependency graphs, and process calculi.

28. Questions for External Reviewers

7. Is tau:T->{+,0,-} the correct formal placement, or should classification operate on G, T, and R separately?
8. What classes of obstruction should be primitive: boundary, contradiction, dependency failure, resource limit, or policy denial?
9. Should relief and bypass be primitive operations or derived operations?
10. What distance metrics best define Delta for governance, runtime, and UOR-relational systems?
11. Does the neutral state 0 map cleanly to existing concepts in planning, scheduling, and process theory?
12. What minimal counterexample would falsify the usefulness of ARCH-1T?

29. Related Work Orientation

The framework should be positioned carefully. It is not a replacement for established models. It is a unifying review layer and operational synthesis. The closest neighbors are term algebras, algebraic data types, state-transition systems, Petri nets, workflow calculi, planning systems, dependency graphs, control theory, and runtime monitoring. The distinctive value of ARCH-1R/T is the explicit lineage from recursive generation to ternary reachability and coherence validation.

30. Final Conclusion

ARCH-1R/T is best understood as a layered operational extension of the original ARCH-1 kernel. The kernel generates possibility. Traversal explores it. Obstruction constrains it. Dependency closure propagates constraint. Ternary classification

distinguishes reachable, latent, and obstructed states. Observation records what happened. Delta measures how well the model matched reality.

The central practical result is the formal protection of the neutral state: 0 != -. In real systems, not yet is frequently mistaken for impossible. ARCH-1T gives that distinction a formal home. That single distinction improves governance, scheduling, dependency analysis, planning, runtime diagnostics, and relational modeling.

Appendix A - Compact Formal Reference

```
Kernel:
  Sigma := mu S.(beta OR Omega(S,S))

Generation:
  G := Closure_Omega(beta)

Traversal:
  rho : G -> T
  T := rho(G)

Obstruction:
  O subseteq T
  O* := closure_D(O)
  R := T \ O*

Classification:
  tau : T -> {+,0,-}

Observation:
  Q := observed outcomes

Validation:
  Delta := d(R,Q)

Quantification proposal:
  sigma : T -> [0,1]
  C : T -> Real_nonnegative
  P_lambda := {x in R | C(x) <= lambda}
```

Appendix B - Glossary

Symbol	Name	Meaning
beta	Anchor	Terminal seed or starting element.
Omega	Constructor	Binary recursive constructor.
mu	Closure	Least fixed-point recursive closure.
G	Generated space	All structurally generated possibilities.
rho	Traversal	Activation, scheduling, planning, or execution operator.
T	Traversed space	Generated states selected or attempted by rho.
O	Direct obstruction	Immediate blocking constraint.
O*	Propagated obstruction	Dependency closure of direct obstruction.
R	Realizable space	Traversed space after propagated obstruction.
tau	Classifier	Ternary state function into +, 0, or -.
Q	Observation	Observed outcomes.
Delta	Coherence error	Distance between predicted realization and observation.